HASHTAG VIRALITY AND ENGAGEMENT ANALYSIS

Prepared By: Tahira Rezaie, Any Das and
Natnael Solomon Gebremichael

OVERVIEW

- Introduction
- Focus Area
- Data Acquisition
- Data Storage
- Data Integration and Data Enrichment
- Data Quality
- Softwear Architecture
- Result and Analysis
- Limitation
- Conclusion
- Future Work

INTRODUCTION

Social media hashtags enhance discoverability and engagement, yet not all achieve viral success. This study explores hashtag trends in Tourism, Cultural Heritage, Food, and Wellness industries to identify key factors influencing engagement and provide insights for optimizing social media strategies.

FOCUS AREA

01 Identifying Top Hashtags – Analyzing trends and engagement patterns to determine the most effective hashtags.

02 Industry-Specific Usage – Examining how sectors like Tourism, Cultural Heritage, Food, and Wellness utilize hashtags for maximum engagement.

DATA ACQUISITION

Data sources

- Instagram Data: Collected in real-time using the Apify API, tracking posts with industry-specific hashtags. It included metrics like likes, shares, replies, and timestamps, stored in MongoDB for easy access.
- Kaggle Dataset: A historical dataset providing structured engagement trends over time. It was manually processed and used for comparative analysis, complementing the real-time data.

Data Collection Process

```python
# Asynchronous route to fetch data
async def fetch_api_data(keyword):
    url = f"https://ensembledata.com/apis/tt/hashtag/recent-posts?name={keyword}&days=90&remap_out
    try:
        response = requests.get(url)
        response.raise_for_status()  # Raise an exception for bad status codes
        return response.json()
    except requests.exceptions.RequestException as e:
        print(f"Error fetching data from {url}: {e}")
        return None
```

Retrieves Instagram data from an external API

```python
#
def fetch_instagram_api_data():
    url = "https://api.apify.com/v2/acts/apify~instagram-scraper/runs/last/dataset/items?token=api
    try:
        response = requests.get(url)
        response.raise_for_status()  # Raise an exception for bad status codes
        return response.json()
    except requests.exceptions.RequestException as e:
        print(f"Error fetching data from {url}: {e}")
        return None
```

Data Prepocessing and Cleaning

To ensure high-quality data, we applied a systematic cleaning process. Missing values were handled to prevent gaps in analysis. Timestamps were standardized for consistency across records. Hashtags were normalized to maintain uniformity in textual data. Duplicate records were identified and removed to avoid redundancy. After these steps, the final dataset was clean, structured, and ready for analysis.

```
data_one_csv = file_path + "/modules/social_media/data_one.csv"
df = pd.read_csv(data_one_csv)
df = df.replace({np.nan: None})
df["Extracted_Hashtags"] = df["Status text"].apply(extract_hashtags)
df["Engagement_Score"] = df["like_count"] + df["retweet_count"] + df["reply_count"]
df["Date1"] = str(pd.to_datetime(df["Date1"], errors="coerce"))
data = df[["Date1", "Extracted_Hashtags", "Engagement_Score"]].to_dict(orient="records")
existing_data = await collection_social_media_filtered_csv.find({}, {"Date1": 1}).to_list(None)
existing_dates = set(item["Date1"] for item in existing_data)
```

DATA STORAGE

Database Management System

We used MongoDB, a NoSQL database, to store and manage social media data due to its scalability and flexibility. Its schema-less structure efficiently handles dynamic, unstructured data, making it ideal for real-time social media analysis. Additionally, MongoDB's Motor library enables asynchronous querying, improving performance in FastAPI.

Data Modelling

We structured social media data into three MongoDB collections: raw API data, historical data from Kaggle, and processed data for analysis. Each collection stores key engagement metrics, including likes, retweets, and hashtags. This approach ensures efficient retrieval and structured storage for social media insights.

```
social_media_router = main_router
collection_social_media_api = database["collection_social_media_api"]
collection_social_media_csv = database["social_media_csv_data"]
collection_social_media_filtered_csv = database["social_media_filtered_csv_data"]
```

```python
async def loadAPI():
    data = fetch_instagram_api_data()
    existing_data = await collection_social_media_api.find({}, {"id": 1}).to_list(None)
    existing_item = set(user["id"] for user in existing_data)

    # Filter out duplicates before inserting
    unique_data = [item for item in data if item["id"] not in existing_item]

    if unique_data:
        result = collection_social_media_api.insert_many(unique_data)
        print("result", result)
        return {"message": "Inserted data"}
```

- Fetches API data and stores it in MongoDB.
- Filters out duplicates before inserting new data.

Uses a FastAPI endpoint to retrieve stored data with pagination.

```python
@social_media_router.get('/get-saved-api-data', tags=['API-Data'])
async def getSavedApiData(page: int = 0, per_page: int = 10):
    data = []
    async for item in collection_social_media_api.find().skip(page).limit(per_page):
        item['_id'] = str(item['_id'])
        data.append(item)
    return getPaginatedResponse(data=data, page=page, per_page=per_page)
```

DATA INTEGRATION AND ENRICHMENT

Integration of Data Sources

We integrated social media data from APIs (Instagram) and CSV files (Twitter) into MongoDB for structured storage and analysis. Using tools like Pandas and FastAPI, we extracted key features such as hashtags and engagement scores. To ensure data accuracy, we handled errors by removing duplicate records and validating hashtag extraction. This resulted in a clean, enriched dataset for further analysis.

We combined structured CSV files with real-time API data to create a unified dataset.

```python
async def loadFilteredCSV():
    # Read CSV data
    data_one_csv = file_path + "/modules/social_media/data_one.csv"
    df = pd.read_csv(data_one_csv)
    # Replace NaN with None
    df = df.replace({np.nan: None})
    # Extract hashtags and calculate engagement
    df["Extracted_Hashtags"] = df["Status text"].apply(extract_hashtags)
    df["Engagement_Score"] = df["like_count"] + df["retweet_count"] + df["reply_count"]
    # Convert Date1 column to datetime format
    df["Date1"] = str(pd.to_datetime(df["Date1"], errors="coerce"))
    # Convert DataFrame to dictionary format for MongoDB
    data = df[["Date1", "Extracted_Hashtags", "Engagement_Score"]].to_dict(orient="records")
    # Get existing data
    existing_data = await collection_social_media_filtered_csv.find({}, {"Date1": 1}).to_list(None)
    existing_dates = set(item["Date1"] for item in existing_data)
    # Filter out duplicates before inserting
    unique_data = [item for item in data if item["Date1"] not in existing_dates]
    if unique_data:
        result = await collection_social_media_filtered_csv.insert_many(unique_data)
        return {"message": "Inserted data", "inserted_count": len(result.inserted_ids)}
    return {"message": "No unique data to insert"}
```

```python
async def saveFilteredAPIData(background_tasks: BackgroundTasks, page: int = 0, per_page: int = 200):
    processed_data = []
    background_tasks.add_task(loadFilteredCSV)
    async for post in collection_social_media_api.find().skip(page).limit(per_page):
        post['_id'] = str(post['_id'])
        extracted_hashtags = []
        total_engagement = 0
        latest_date = None
        for comment in post.get('latestComments', []):
            comment['id'] = str(comment['id'])
            comment['Date1'] = post.get('latestComments', [])[0]['timestamp']
            hashtags = re.findall(r"#\w+", comment.get('text', ''))
            print("extracted_hashtags.extend(hashtags)", extracted_hashtags.extend(hashtags))
            extracted_hashtags.extend(hashtags)
            total_engagement += comment.get('repliesCount', 0) + comment.get('likesCount', 0)
            timestamp = comment.get('timestamp')
            if timestamp:
                latest_date = post.get('latestComments', [])[0]['timestamp']
        response = {
            "_id": post.get('_id', 'unknown_id'),
            "Date1": latest_date,
            "Extracted_Hashtags": list(set(extracted_hashtags)),
            "Engagement_Score": total_engagement + len(extracted_hashtags)
```

- Retrieves and processes social media posts, checks for duplicates, and stores only new records.
- Extracts hashtags, calculates engagement scores, and determines the latest timestamp to enhance data insights.

```python
    processed_data.append(response)
existing_data = await collection_social_media_filtered_csv.find({}, {"_id": 1}).to_list(None
existing_dates = set(item["_id"] for item in existing_data)
unique_data = [item for item in processed_data if item["_id"] not in existing_dates]
if unique_data:
    result = await collection_social_media_filtered_csv.insert_many(unique_data)
    return {"message": "Data Integrated",
            }
else:
    return {"message": "No new data available for integration",
            }
```

DATA QUALITY

We implemented an automated data quality check to identify missing fields in our dataset. The system scans for required attributes and flags any missing values, ensuring data completeness and reliability.

Overview

Brought to you by YData

Overview | Alerts 3 | Reproduction

Dataset statistics

Number of variables	3
Number of observations	100
Missing cells	0
Missing cells (%)	0.0%
Duplicate rows	0
Duplicate rows (%)	0.0%
Total size in memory	2.5 KiB
Average record size in memory	25.3 B

Variable types

Text	1
DateTime	1
Numeric	1

```python
@social_media_router.post("/check_data_quality")
async def check_data_quality():
    try:
        fields_to_check = ["_id", "Extracted_Hashtags", "Engagement_Score", "Date1"]  # List of fields
        missing_fields = []

        for field in fields_to_check:
            missing_count = await collection_social_media_filtered_csv.count_documents(
                {field: {"$exists": False}})  # await the async function
            if missing_count > 0:
                missing_fields.append(field)

        if missing_fields:
            return {"status": "failed",
                    "message": f"Documents are missing the following fields: {', '.join(missing_fields)}"}
        else:
            return {"status": "passed", "message": "All documents have the required fields."}

    except Exception as e:
        raise HTTPException(status_code=500, detail=f"Data quality check failed: {e}")
```

SOFTWARE ARCHITECHTURE

- Modular Microservices Design for scalability and flexibility.
- Data Collection Module gathers Instagram data (Apify API) and historical trends (Kaggle).
- Processing & Storage Module integrates and stores data in MongoDB.
- API Layer enables querying and analysis using FastAPI.
- Tech Stack: Python, FastAPI, MongoDB, Pandas, NumPy.

RESULTS AND ANALYSIS

TOP HASHTAGS BY ENGAGEMENT

- Hashtags related to tourism, such as #BiharTourism and #BlissfulBihar, saw the highest engagement, indicating strong public interest.
- Short and region-specific hashtags performed better, with #BiharTourism leading at 10,791 engagements.
- General hashtags like #Bihar and #BiharKiShaan had lower engagement, showing that specificity boosts interaction.


```
{
  "message": "Data get successfully",
  "data": [
    {
      "hashtag": "#BiharTourism",
      "total_engagement": 10791
    },
    {
      "hashtag": "#BlissfulBihar",
      "total_engagement": 9963
    },
    {
      "hashtag": "#Bihar",
      "total_engagement": 2913
    },
    {
      "hashtag": "#BiharKiShaan",
      "total_engagement": 2774
    },
    {
      "hashtag": "#RangBiharKe",
      "total_engagement": 1927
    },
```

INDUSTRY-SPECIFIC TRENDS

- Tourism leads with 725,943 engagements, making it the most impactful sector.
- Travel, Cultural, and Food sectors show moderate engagement, influenced by trends and events.
- Wellness struggles with 2,054 engagements, appealing to a small but dedicated audience.


```
{
  "industry_engagement": {
    "tourism": 725943,
    "travel": 90540,
    "cultural": 11076,
    "food": 11760,
    "wellness": 2054
  }
}
```

LIMITATIONS

01 Platform-Specific Data: Insights are based only on Instagram and Kaggle, missing engagement trends from other platforms.

02 Lack of Demographic Data: User interactions couldn't be analyzed based on age, location, or preferences.

03 Temporal Constraints: The study covers a limited time frame, potentially missing long-term trends.

CONCLUSION

This study analyzed hashtag virality across Tourism, Cultural, Food, and Wellness sectors, finding that thematic relevance, length, and timing drive engagement. Hashtags tied to cultural events, seasonal trends, and health movements performed best. While offering actionable insights, the study is limited by its reliance on public data and a single platform (Twitter).

FUTURE WORK

- Cross-Platform Analysis: Expand to platforms like Facebook, Instagram, and TikTok to compare hashtag virality.
- Demographic Data: Incorporate age, gender, and location data to understand user-specific engagement.
- Predictive Analytics: Use machine learning to forecast hashtag virality.
- Real-Time Event Impact: Study how global crises or viral news affect hashtag trends.

THANK YOU